PEX 프로토콜 분석 문서

개요

PEX(Peer Exchange)는 BEP 11에 정의된 BitTorrent 확장 프로토콜로, 피어들이 서로 다른 피어의 정보를 교환할 수 있게 해줍니다. Rain BitTorrent 클라이언트는 완전한 PEX 구현을 제공하여 트래커나 DHT 없이도 피어 발견이 가능하도록 합니다.

주요 특징

- BEP 11 PEX 프로토콜 완전 구현
- 1분 간격으로 피어 정보 교환
- 최대 50개 피어 정보 전송 제한 (BEP 11 준수)
- 프라이빗 토렌트에서는 PEX 비활성화
- 최근 연결된 피어 추적으로 초기 PEX 리스트 구성

PEX 확장 메시지 구현

1. PEX 메시지 구조

PEX 메시지는 BitTorrent 확장 프로토콜을 통해 전송됩니다 (`internal/peerprotocol/extension.go`):

```
// ExtensionPEXMessage는 PEX 확장을 위한 메시지입니다.
type ExtensionPEXMessage struct {
Added  string  `bencode:"added"`  // 새로 추가된 피어들 (compact 형식)
```

```
Dropped string `bencode:"dropped"` // 연결이 끊어진 피어들 (compact 형식)
}
```

2. 확장 프로토콜 등록

PEX는 확장 핸드셰이크에서 등록됩니다:

```
const (
ExtensionIDPEX = 2 // PEX 확장 메시지 ID
)

const (
ExtensionKeyPEX = "ut_pex" // PEX 확장 키
)

// 확장 핸드셰이크 메시지에 PEX 등록
func NewExtensionHandshakeMessage(version string) ExtensionHandshakeMessage {
return ExtensionHandshakeMessage{
M: map[string]uint8{
ExtensionKeyMetadata: ExtensionIDMetadata,
ExtensionKeyPEX: ExtensionIDPEX, // PEX 확장 등록
},
V: version,
}
```

```
}
```

3. PEX 메시지 파싱

확장 메시지 파싱 시 PEX 메시지를 처리합니다:

```
func (m *ExtensionMessage) UnmarshalBinary(data []byte) error {
// ... 기본 파싱 로직 ...
switch m.ExtendedMessageID {
case ExtensionIDPEX:
var extMsg ExtensionPEXMessage
err = dec.Decode(&extMsg)
m.Payload = extMsg
}
return nil
}
```

피어 정보 교환 과정

1. PEX 구조체

각 피어는 개별 PEX 인스턴스를 가집니다 (`internal/peer/pex.go`):

```
type pex struct {
conn *peerconn.Conn // 피어 연결
extID uint8 // PEX 확장 메시지 ID
```

```
// 추가/제거된 피어 정보를 담는 리스트
pexList *pexlist.PEXList
// 피어 추가/제거 신호를 받는 채널
pexAddPeerC chan *net.TCPAddr
pexDropPeerC chan *net.TCPAddr
closeC chan struct{}
doneC chan struct{}
}
```

2. PEX 인스턴스 생성

피어가 PEX를 지원한다고 알려오면 PEX 인스턴스를 생성합니다:

```
func newPEX(conn *peerconn.Conn, extID uint8, initialPeers map[*Peer]struct{}
// 최근 본 피어들로 초기 dropped 리스트 구성
pl := pexlist.NewWithRecentlySeen(recentlySeen.Peers())
// 현재 연결된 피어들을 added 리스트에 추가
for pe := range initialPeers {
if pe.Addr().String() != conn.Addr().String() {
pl.Add(pe.Addr())
}
}
return &pex{
conn: conn,
```

```
extID: extID,
pexList: pl,
pexAddPeerC: make(chan *net.TCPAddr),
pexDropPeerC: make(chan *net.TCPAddr),
closeC: make(chan struct{}),
doneC: make(chan struct{}),
}
}
```

3. PEX 시작

확장 핸드셰이크에서 PEX 지원이 확인되면 PEX를 시작합니다 (`torrent_messagehandler.go`):

```
func (t *torrent) handleExtensionHandshake(pe *peer.Peer, msg peerprotocol.Exten
// ... 기타 확장 처리 ...
if t.session.config.PEXEnabled {
if _, ok := msg.M[peerprotocol.ExtensionKeyPEX]; ok {
// 프라이빗 토렌트가 아닌 경우에만 PEX 시작
if t.info != nil && !t.info.Private {
pe.StartPEX(t.peers, &t.recentlySeen)
}
}
}
```

```
}
```

4. PEX 메인 루프

PEX는 별도 고루틴에서 실행됩니다:

```
func (p *pex) run() {
defer  close(p.doneC)
// 초기 피어 정보 전송
p.pexFlushPeers()
// 1분 간격으로 피어 정보 전송
ticker := time.NewTicker(time.Minute)
defer  ticker.Stop()
for {
select {
case  addr := <-p.pexAddPeerC:
// 새 피어 추가
p.pexList.Add(addr)
case  addr := <-p.pexDropPeerC:
// 피어 연결 해제
p.pexList.Drop(addr)
case <-ticker.C:
// 주기적으로 피어 정보 전송
p.pexFlushPeers()
```

```
case <-p.closeC:
return
}
}
}
```

PEX 메시지 처리 로직

1. 피어 정보 전송

1분마다 또는 즉시 피어 정보를 전송합니다:

```
func (p *pex) pexFlushPeers() {
// PEX 리스트에서 added/dropped 피어 정보 가져오기
added, dropped := p.pexList.Flush()
if  len(added) == 0 && len(dropped) == 0 {
return  // 전송할 정보가 없으면 종료
}
// PEX 확장 메시지 생성
extPEXMsg := peerprotocol.ExtensionPEXMessage{
Added: added, // compact 형식의 피어 주소들
Dropped: dropped, // compact 형식의 피어 주소들
}
// 확장 메시지로 래핑
```

```
msg := peerprotocol.ExtensionMessage{
ExtendedMessageID: p.extID,
Payload: extPEXMsg,
}
// 피어에게 전송
p.conn.SendMessage(msg)
}
```

2. PEX 메시지 수신 처리

다른 피어로부터 PEX 메시지를 받으면 새로운 피어들을 처리합니다:

```
func (t *torrent) handleExtensionMessage(pe *peer.Peer, msg peerprotocol.Extensi
switch  msg := msg.Payload.(type) {
case peerprotocol.ExtensionPEXMessage:
if !t.session.config.PEXEnabled {
break
}
// Added 피어들 파싱
addrs, err := tracker.DecodePeersCompact([]byte(msg.Added))
if  err != nil {
break
}
// PEX 소스로 새 피어들 처리
```

```
t.handleNewPeers(addrs, peersource.PEX)

// Dropped 피어들 파싱 (현재는 정보만 파싱)

addrs, err = tracker.DecodePeersCompact([]byte(msg.Dropped))

if  err != nil {

break

}

t.handleNewPeers(addrs, peersource.PEX)

}

}
```

피어 소스 관리

1. PEX 피어 소스 분류

PEX를 통해 발견된 피어는 별도로 분류됩니다 (`internal/peersource/peersource.go`):

```
const (

Tracker Source = iota

DHT

PEX  // PEX를 통해 발견된 피어

Manual

Incoming

)
```

2. 토렌트 레벨 PEX 관리

토렌트는 PEX 이벤트를 모든 PEX 지원 피어에게 전파합니다:

```
// 새 피어 연결 시 모든 PEX 피어에게 알림
func (t *torrent) pexAddPeer(addr *net.TCPAddr) {
for pe := range t.peers {
if pe.PEX != nil {
pe.PEX.Add(addr)
}
}
}

// 피어 연결 해제 시 모든 PEX 피어에게 알림
func (t *torrent) pexDropPeer(addr *net.TCPAddr) {
for pe := range t.peers {
if pe.PEX != nil {
pe.PEX.Drop(addr)
}
}
}
```

3. 피어 연결/해제 이벤트 처리

피어 연결과 해제 시 PEX 이벤트가 발생합니다:

```go
// 피어 연결 시 (torrent_peer.go)
func (t *torrent) handleNewPeer(conn net.Conn, peerID [20]byte, extensions peerp
addr := conn.RemoteAddr().(*net.TCPAddr)
t.pexAddPeer(addr) // PEX에 새 피어 추가 알림
// ... 피어 처리 로직 ...
if  duplicateID {
t.pexDropPeer(addr) // 중복 ID면 PEX에 제거 알림
return
}
}


// 피어 연결 해제 시 (torrent_close.go)
func (t *torrent) closePeer(pe *peer.Peer) {
t.pexDropPeer(pe.Addr()) // PEX에 피어 제거 알림
// ... 기타 정리 로직 ...
}
```

PEXList 구현 상세

1. PEXList 구조

PEX 리스트는 추가/제거된 피어를 관리합니다 (`internal/pexlist/pexlist.go`):

```go
type PEXList struct {
added  map[tracker.CompactPeer]struct{} // 추가된 피어들
dropped  map[tracker.CompactPeer]struct{} // 제거된 피어들
flushed  bool  // 첫 전송 여부
}
```

2. 피어 추가/제거

```go
// 피어 추가 (dropped에서 제거하고 added에 추가)
func (l *PEXList) Add(addr *net.TCPAddr) {
p := tracker.NewCompactPeer(addr)
l.added[p] = struct{}{}
delete(l.dropped, p) // 중복 방지
}

// 피어 제거 (added에서 제거하고 dropped에 추가)
func (l *PEXList) Drop(addr *net.TCPAddr) {
peer := tracker.NewCompactPeer(addr)
l.dropped[peer] = struct{}{}
delete(l.added, peer) // 중복 방지
}
```

3. 피어 정보 플러시

BEP 11 규격에 따라 최대 50개 피어만 전송합니다:

```
const maxPeers = 50 // BEP 11: 최대 50개 피어 제한


func (l *PEXList) Flush() (added, dropped string) {
added = l.flush(l.added, l.flushed)
dropped = l.flush(l.dropped, l.flushed)
l.flushed = true
return
}


func (l *PEXList) flush(m map[tracker.CompactPeer]struct{}, limit bool) string
count := len(m)
if limit && count > maxPeers {
count = maxPeers // 50개 제한 적용
}
var s strings.Builder
s.Grow(count * 6) // 각 피어는 6바이트 (IP 4바이트 + Port 2바이트)
for p := range m {
if count == 0 {
break
}
count--
```

```go
// compact 형식으로 직렬화
b, err := p.MarshalBinary()
if err != nil {
panic(err)
}
s.Write(b)
delete(m, p) // 전송 후 제거
}
return s.String()
}
```

최근 피어 추적

1. RecentlySeen 구조

최근 연결된 피어들을 추적하여 초기 PEX 리스트를 구성합니다:

```go
const MaxLength = 25 // 최대 25개 피어 추적

type RecentlySeen struct {
peers []tracker.CompactPeer
offset int // 순환 버퍼 오프셋
length int // 현재 길이
}
```

2. 피어 추가

```
func (l *RecentlySeen) Add(addr *net.TCPAddr) {
cp := tracker.NewCompactPeer(addr)
// 중복 체크
if l.has(cp) {
return
}
// 순환 버퍼로 관리
if l.length >= MaxLength {
l.peers[l.offset] = cp
} else {
l.peers = append(l.peers, cp)
l.length++
}
l.offset = (l.offset + 1) % MaxLength
}
```

3. 초기 PEX 리스트 구성

새 PEX 인스턴스 생성 시 최근 피어들을 dropped 리스트에 추가:

```
func NewWithRecentlySeen(rs []tracker.CompactPeer) *PEXList {
l := New()
```

```go
for _, cp := range rs {
l.dropped[cp] = struct{}{} // 최근 피어들을 dropped으로 추가
}
return l
}
```

PEX 통계 및 모니터링

1. PEX 피어 통계

PEX를 통해 발견된 피어 수를 추적합니다:

```go
// torrent_stats.go
type Stats struct {
Addresses struct {
Total int
Tracker int
DHT int
PEX int // PEX를 통해 발견된 피어 주소 수
}
}

func (t *torrent) Stats() Stats {
// ...
```

```
s.Addresses.PEX = t.addrList.LenSource(peersource.PEX)

// ...

}
```

2. RPC API 통계

RPC API를 통해 PEX 통계를 조회할 수 있습니다:

```
// session_rpc_handler.go

func (h *rpcHandler) GetSessionStats(args *GetSessionStatsArgs, reply *GetSessio

// ...

reply.Stats.Addresses.PEX = s.Addresses.PEX

// ...

}
```

성능 고려사항

1. 메모리 효율성

- 순환 버퍼로 최근 피어 추적 (최대 25개)
- PEX 리스트는 전송 후 즉시 정리
- Compact 피어 형식으로 메모리 사용량 최소화

2. 네트워크 효율성

- 1분 간격으로 배치 전송

- BEP 11 준수로 최대 50개 피어만 전송
- 중복 피어 정보 전송 방지

3. 프라이빗 토렌트 보호

```
// 프라이빗 토렌트에서는 PEX 비활성화
if t.info != nil && !t.info.Private {
pe.StartPEX(t.peers, &t.recentlySeen)
}
```

관련 문서

선행 지식

- 04-01-networking-packages-analysis.md - 피어 연결과 메시지 처리 기초
- 01-system-architecture.md - PEX가 전체 시스템에서 차지하는 위치

관련 주제

- 05-01-dht-implementation-analysis.md - 다른 피어 발견 메커니즘과의 비교
- 06-02-concurrency-analysis.md - PEX 고루틴 관리

후속 학습

- 06-01-performance-optimization-analysis.md - PEX 성능 최적화
- 07-01-library-usage-guide.md - PEX 설정 및 사용법

> **네비게이션:** 이전: DHT 구현 분석 | 목차 | 다음: MSE 암호화 분석